AI Security Pipeline: Prompt Injection Detection

Project Workbook

By

Dao Seckar (dao.seckar@sjsu.edu)
Haroon Razzack (haroon.razzack@sjsu.edu)
Kyle Glover (kyle.glover@sjsu.edu)
Nkono Andrew Mwase (nkonoandrew.mwase@sjsu.edu)

03/05/2026

Advisor: Wencen Wu

Chapter 1. Literature Search, State of the Art

Literature Search

1.1 The Prompt Injection Threat in LLM-Integrated Applications

As of recently, prompt injection is a popular topic within the LLM and AI community. One of the other topics that is not as popular but closely related to it would be indirect prompt injection as mentioned by Greshake et al. [2]. While direct Prompt Injection requires the user themselves to inject harmful and malicious input into the LLM, indirect prompt injection involves an agent tooling being manipulated without the knowledge of the user by a malicious attacker in order to gain private information, data, and emails. The way LLM works is by scanning and reading the entire content then generates output in which there is little safeguard against instructions that are harmful to its output. As demonstrated by Greskate et al. systems like ChatGPT and Bing are prone to these kind of attacks where they will end up scanning a hidden prompt in an image or email or webpage that have been deliberately put in by the malicious attacker in order to get the AI to hand over information that should not have been given [2].

1.2 Attacks Against Tool-Using LLM Agents

While LLM agents on websites are mostly prone to leaking information or chat logs of users with the agents, tool-using LLM Agents are also prone to inflicting real-world damages if taken advantage of. When a LLM Agent is integrated into more than just a webpage, it now also has the permission and power to perform real world tasks such as money transaction, system takeover, emails access, and tools inside of the user machine's system. Greshake et al. [2] demonstrate that with careful planning and content poisoning, the attacker was able to gain remote access to LLM agents and compromise the user's system without ever interacting with the user. Zhan et al. [6] showed examples that supported this narrative by conducting many experiments regarding attackers manipulating the AI agents to use tools that help them gain access to the user's personal appointments. This inspired the creation of the allow-list tool based gateway which is designed to prevent these kinds of attacks from malicious attackers.

1.3 Detection and Defense Mechanisms

Current defense research can be grouped into prevention, detection, and mitigation/control layers [3,10,11]. Prevention-oriented techniques such as instruction delimiting, sandwiching, and prompt hardening reduce attack success in some settings but are bypassable under adaptive attacks [4,11]. Detection-focused approaches like classifier-based methods and LLM-as-judge pipelines improve sensitivity but run into false positives, domain shift, and multi-turn reliability issues [11,12,17,21,24].

1.3.1 Prevention and Detection

Yi et al. [11] created the BIPIA Benchmark with approximately 710 test samples across email, web, table, and code scenarios. They evaluated multiple prevention strategies on the BIPIA benchmark, including border-string delimiting, re-stating instructions after untrusted content, and in-context learning with few-shot resistance examples. None eliminated the threat: border-string defenses reduced attack success rates by only 10-20 percentage points, and perplexity-based detection improved sensitivity but produced high false positive rates that would significantly degrade utility in real deployments [11].

Chen et al. advanced this line of work by combining detection with active removal, segmenting documents to isolate injected instructions and training the model to extract them before processing. Their detect-then-remove pipeline reduced attack success while maintaining task utility [12].

1.3.2 Model-level defenses

Two important model-level defense lines are worth describing:

- Instruction hierarchy: improves instruction-priority behavior and reduces many attack outcomes, however gains vary by the attack type and do not remove risk for indirect or context heavy settings.
- Defensive Tokens: shows strong benchmark improvements with low utility costs under testing. However, it remains model or benchmark dependent instead of a guarantee [14].

1.4 Evaluation Benchmarks and Metrics

One such benchmark for evaluating an agent's susceptibility to indirect prompt injection (IPI) is InjecAgent, proposed by Zhan et al. [6]. Utilizing 1,054 test cases across 17 user tools, Zhan et al. demonstrated a 24% attack success rate (ASR) in ReAct-based open-source GPT-4 agents and a ~47% ASR in GPT-3.5 agents, with rates nearly doubling when introducing a reinforced hacking prompt. Safety prompts, which restrict agent usage of tools that could be exploited by injection attempts, managed to reduce ASR to ~15% in GPT-4, rendering IPI less likely but still a substantial threat. With such a study pertaining to tool-based agents, a high risk rate for IPI is expected at this time.

The systemic framework BIPIA presented by Yi et al. [11] experimented upon 25+ LLMs, all of which were vulnerable to IPI, with ASR's exceeding 50%, climbing as high as 90% in some cases all across multiple defense benchmarks. These experiments demonstrate that any LLM is capable of exploitation, and IPI and prompt injection exploits cannot be fully mitigated. Thus, risk mitigation should be ideally as close to 0% as possible.

However, it is important to consider model utility alongside security. As Debenedetti et al. note in their presentation of AgentDojo, no model achieves both high utility and high security simultaneously [20], and the trade-off between the two must be carefully considered. Such a benchmark is best measured through dynamic task generation as opposed to static environments, so as to allow a flexible environment for the model and

allows a more robust evaluation of model performance and security [20]. Alternatively, DefensiveTokens could be considered as a benchmark to strive for, as they significantly reduce ASR to the upwards of 50%, and can be adaptively applied to potentially malicious inputs [14], sacrificing performance only when attacks or risks are suspected rather than reducing performance for all inputs.

1.5 Existing Tools and Frameworks

Although academic benchmarks establish theoretical vulnerability, deployed systems come from it in production. Analysis of major agentic coding tools such as Claude Code, Cursor, OpenCode, and OpenHands reveals that the threat model described by Greschke et al. and formalized by Liu et al. is not hypothetical. It maps directly onto real CVEs, real architectural failures, and real design pivots made by engineering teams at major companies.

The CVE record for Cursor is particularly interesting. Between mid-2025 and the release of sandboxing features, six critical vulnerabilities were documented, all following the same chain: prompt injection into a trusted content source, like a read me MCP response or a code comment causes the agent to overwrite a configuration file. Like the MCP.json, the.code workspace, the CLI config, all of which the IDE auto-loads and executes as trusted instructions. [27, 28] CVE 2025-54135 demonstrated full remote code execution via a Poison Slack message processed by an MCP connected agent [28]. Thus the immense risk of unwarranted MCP injection is significant even in 2026 as MCPs become more mainstream.

Open Code presents a complimentary and arguably more severe case. CVE-2026-22812 documented an unauthenticated local HTTP server that accepted arbitrary command execution from any process or browser tab on the same machine. This is a textbook example of OWASP's excessive agency risk. Beyond the CVE, Open Code's architecture auto-loads opencode.json configs and runs `bun install` in opencode directories at startup without any workspace trust verification. Meaning a user who clones a booby-trap repository and runs the tool is immediately compromised [34,35]. Moreover, OpenCode has no built-in OS-level sandboxing, which means that its permission system operates entirely at the application layer and can be bypassed via Bash commands [34]. This makes OpenCode the clearest available example of the consequences of what happens when a tool relies solely on application-level permission and does not contain any hard enforcement boundaries.

The second consistent finding across deployed tools is the documented failure of approval prompt-based security. Multiple independent sources identify approval fatigue as a real and exploitable condition. Developers begin rubbing rubber-stamping confirmations after encountering enough of them, such that attackers can explicitly frame dangerous actions as routine in the approval prompt itself. Cursor's engineering team states that sandboxed agents "stop 40% less often" than unsandboxed ones. NVIDIA's AI Red Team identifies "user habituation" as a reason manual approval cannot be the primary control [31]; and OpenHands describes confirmation mode as a "stopgap solution" that limits agent value [30]. This multi-source convergence directly validates

the project's thesis that deterministic enforcement must replace or anchor probabilistic human approval.

The Model Context Protocol (MCP) has emerged as a distinct new attack surface within this period. Because MCP tool descriptors are natural-language strings that the LLM follows as instructions, a single malicious or compromised MCP server can redirect agent behavior, exfiltrate credentials, and persist backdoors, without any signing, verification, or integrity check on the tool definition itself [26,28]. This expands the scope of Boundary A in the project's trust model: MCP server responses must be treated as untrusted content and pass through risk scoring before entering agent context, the same as any other retrieved data.

Across all deployed tools, a convergent architectural pattern is visible: model-level defense (probabilistic, LLM-based) feeds into a permission/policy layer (pattern matching or rules), which feeds into an OS-level sandbox (deterministic, kernel-enforced) [25,26,30]. This is the same layered structure the academic literature argues for. The gap the project addresses is that no current tool implements a dedicated risk-scored gateway between the policy layer and execution, permissions are binary (allow/deny patterns), and the sandbox is the only hard boundary. The project proposes an explicit, context-aware enforcement component that no deployed system currently provides [25,26,29,31].

State-of-the-Art Summary

The state-of-the-art security for tool-using LLMs has advanced rapidly in threat understanding and benchmark quality. However, there is no agreement in a broadly adopted deterministic enforcement architecture within different methodologies.

When it comes to the threats side, the field has transitioned away from anecdotal jailbreak examples to formal models and realistic agent benchmarks. Foundational work established indirect prompt injection and the confused deputy problem in LLM-integrated systems. Formal benchmarking then demonstrated that no single defense category fully solves the problem across different models and tasks. Newer benchmark ecosystems like Inject Agent, Agent Dojo, and ASB confirm that vulnerability persists when agents can call tools, with substantial attack success under realistic interaction settings. [6,7,20].

On the defense side, leading approaches include: Prompt hardening, structured instruction prioritization, trained classifiers and hybrid detect-then-remove pipelines. [11,12,13,14] These methods materially improve robustness, but the consensus result is partial mitigation rather than elimination. Two recurring bottlenecks are high false positive pressure and performance decline in context-dependent or multi-turned tasks [5,11,17]. This is likely why current literature frames security as more of a systems problem, and not a model training problem.

In deployed production ecosystems, commercial and open-source guardrail tools provide valuable controls. [21,22,23,24] However, they typically stop at detection or advisory

policy layers for high agency settings, creating a practical gap between the "flagging risk" and "blocking unsafe side effects."

Analysis of major agentic coding tools (i.e., Claude Code, Cursor, Open Code, Open Hands) confirms that this gap exists in production. Moreover, all documented CVEs for these systems follow the same injection-to-configuration-overwrite-to-RCE chain predicted by foundational research, and all teams independently converged on layered defense architectures with OS-level sandboxing as the primary hard boundary.

Two additional findings from industry sources are directly relevant. Firstly, approval prompt base security has shown to fail at scale due to approval fatigue.Approval fatigue is the concept that developers tend to rubber-stamp confirmations from agents due to the duress of cognitive load, and this failure mode is exploitable by design. Secondly, the model context protocol (MCP) has introduced a new supply chain attack surface. Tool descriptors are natural language strings with no signing or integrity verification, enabling a single malicious MCP server to redirect agent execution and establish backdoors.

For this particular project, both the academic literature and the deployed systems in production are evidence that support the same thesis. Model-level and detector-level defenses are necessary but must terminate in deterministic execution control for tool calls. This architecture directly addresses the dominant failure modes identified across foundational papers, benchmark studies, real-world CVEs, and practitioner analysis.

Chapter 2. Project Justification

Large Language Model (LLM) agents with tool-using capabilities are rapidly being integrated into production systems, creating critical security vulnerabilities that current defense mechanisms fail to adequately address. This project develops a policy-mediated security pipeline that enforces deterministic controls at the tool execution boundary, addressing the gap between threat detection and action prevention in LLM-integrated applications.

Prompt injection has been identified as the highest-priority threat in LLM-integrated systems [34]. Specifically, indirect prompt injection where adversarial instructions are embedded in external data sources that LLMs retrieve at runtime allows attackers to compromise real-world applications including search engines, code assistants, and RAG pipelines [35]. The fundamental vulnerability stems from the "confused deputy" problem: LLMs cannot distinguish between legitimate system instructions and adversarial content embedded in untrusted data sources [35]. In tool-using agents, this creates an execution-control issue with real-world consequences, not merely a response-quality problem.

Current approaches rely predominantly on detection-based defenses using prompt filtering, input sanitization, and LLM-based guardrails. However, systematic evaluation of existing defenses demonstrates that all prompt-based security mechanisms are fundamentally insufficient [36][37]. Liu et al. [36] evaluated 10 defenses across multiple LLMs and concluded that no single defense eliminates prompt injection. The HackAPrompt competition, with 600,000+ adversarial prompts from 2,800+ participants, proved that every prompt-based defense layer can be eventually broken [37]. This creates an asymmetric advantage for attackers: defense and attack operate in the same communication channel (natural language), inherently favoring adversaries.

This project implements a policy-mediated security pipeline that provides deterministic enforcement at the tool execution boundary. The architecture consists of four core components:

1. Content Normalizer: Standardizes ingested content from multiple sources

2. Risk Engine: Multi-layer threat assessment combining rule-based detection, ML classification, and obfuscation detection

3. Policy Engine: Deterministic decision-making that maps risk scores to explicit actions (ALLOW, SANITIZE, QUARANTINE, REQUIRE_APPROVAL, BLOCK)

4. Tool Gateway: Mandatory enforcement boundary that mediates all tool invocations before execution

This approach enforces strict architectural separation between untrusted input and tool execution, directly implementing recommendations from foundational research [35]. Unlike probabilistic defenses, the gateway ensures no tool runs without policy approval.

The project fills three gaps: lack of enforcement points (detection alone is not enough), insufficient separation of trust boundaries (enabling the confused deputy issue [35]), and missing deterministic policy layers (current LLM-based guardrails are vulnerable to adversarial prompts [37]).

It addresses the top OWASP LLM threats, prompt injection (LLM01) and excessive agency (LLM06), by enforcing policy-driven boundaries [38].

This pipeline will reduce unauthorized tool use, enable auditable decisions, support nuanced risk responses, and allow reproducible security evaluation, bridging detection research with deployable, enforceable defenses.

Chapter 3. Identify Baseline Approaches

3.1 Overview and Objective

Identify state-of-the-art baseline methods and existing implementations that can be compared to or compared against the proposed policy-mediated pipeline.

The baseline selected in this chapter were chosen to reflect the range of practical defenses currently available for LLM systems. Rather than surveying every single proposed defense in the literature, the selection prioritizes tools that share our deployment assumption, which is an agent operating with live tool access and exposure to untrusted content of a particular type of data and that also has public implementations against which we may compare quantitatively.

A pattern observed across all five baselines selected is that they approach security through probabilistic detection (i.e., inputs or outputs are scored, flagged, or re-routed based on classifier confidence). None of the baselines enforce structural restrictions on tool invocation at the policy level. This is precisely the gap, and the policy-mediated pipeline is designed to close this gap and address these issues.

3.2 baseline selection strategy

Baselines will be selected based upon four main criteria:

1. Relevance to prompt injection defense in LLM integrated applications
2. Availability of implementation, either open source code or usable APIs
3. The practicality of comparing with our threat model
4. Feasibility constraints of the team capacity and timeline

Baseline	Type	Implementation Availability	Why Included	Known Limitation
LLM Guard	Detection toolkit (local scanners + PI classifier)	Open-source (protectai/llm-guard)	Strong open-source input/output scanning baseline	Detection-only, no deterministic tool mediation
Rebuff	Multi-layer detection (heuristics + LLM judge + vector similarity)	Open-source (protectai/rebuff)	Useful detection architecture contrast	LLM-as-judge latency/cost and no policy gateway

NeMo Guardrails	Programmable guardrail runtime (Colang + rails)	Open-source (NVIDIA/NeMo-Guardrails)	Closest architecture-style comparison	Probabilistic judge-heavy control; limited deterministic gateway behavior
Lakera Guard	Commercial PI detection API	API product (closed model internals)	Industry-grade detection benchmark	Closed-source; external API dependency; detection-only
Guardrails AI	Validation framework with community PI validators	Open-source (guardrails-ai/guardrails)	Output-validation pattern baseline	Security not primary; uneven PI validator quality

3.4 Baseline from Literature (paper method level)

Method	Source	Implementation Status	Chapter 2/6 Use
StruQ-style structured defense	Chen et al. (USENIX 2025 lineage)	Partial reproducibility likely; but may be heavy	Reference robustness target and compare conceptually
SecAlign preference-optimization defense	CCS 2025 line	Potentially heavy for timeline	Discuss as training-time defense benchmark
Instruction Hierarchy	Wallace et al. 2024	No full public production stack	Compare as model-level defense strategy
Detect-then-remove pipeline	ACL 2025 indirect PI paper	Code available; integration possible with effort	Candidate extension for SANITIZE path

Chapter 4. Dependencies and Deliverables

4.1 Project Dependencies

The Agentic Security Pipeline project relies on several critical dependencies whose absence would halt implementation progress. This section describes only those dependencies that represent genuine showstoppers for the project rather than listing all technical requirements.

The most fundamental dependency is a stable Python 3.11+ development environment with a consistent package manager and reproducible setup mechanism. This dependency is critical because all five pipeline modules normalization, risk scoring, policy decision, tool gateway, and audit logging must execute reliably across all team members' machines and in the deployment environment. Without this consistency, the team cannot validate that tests pass uniformly, cannot reproduce bugs reported by other members, and cannot establish a stable integration baseline. The mitigation strategy involves freezing the environment specification in a requirements.txt file and establishing clear setup documentation in Week 1 of the sprint schedule. This ensures that every team member can replicate the exact runtime environment from the first day of implementation.

The second critical dependency is the establishment of shared interface contracts across all modules in the pipeline. The success of the integration depends entirely on agreed input and output schemas that all modules respect. Since the pipeline operates as a sequential flow where each module consumes the output of the previous stage, any misalignment in data structures or API contracts would cause cascading failures throughout the system. Without locked interface contracts, the team faces continuous rework as mismatches surface during integration, schedule slips due to re-negotiation of module boundaries, and potential integration failures that cannot be localized to a specific module. The mitigation strategy includes implementing a contract freeze gate at the end of Week 1, enforcing that all interface signatures are documented and signed off by all team members before any module implementation begins, and versioning those contracts explicitly so changes can be tracked.

The third dependency is a core test scenario dataset containing representative benign, suspicious, and malicious inputs. This dataset is essential for validating that the risk scoring module correctly identifies threats, that the policy engine makes appropriate decisions, and that the tool gateway enforces those decisions. Without this dataset, there is no objective way to measure progress or verify the correctness of any component. The risk is that team members build modules based on subjective assumptions about what constitutes an attack, leading to divergent implementations that fail when integrated. The mitigation approach involves starting with a minimal eight-scenario pack in Week 1 covering the three required demonstration paths benign allow, suspicious quarantine, and malicious block and then expanding iteratively as testing uncovers edge cases. This ensures that evaluation can begin immediately even with limited coverage, and the dataset grows organically as real weaknesses are discovered.

The fourth critical dependency is the tool gateway stub contract and allowlist mechanism. The entire security model depends on the gateway acting as the sole enforcement boundary for privileged operations. If the gateway cannot be implemented or integrated correctly, the project loses its core value proposition: demonstrating hard mediation of tool calls. Without a functioning gateway, the team cannot show policy enforcement on tool actions, cannot demonstrate approval workflows, and cannot prove that the security pipeline actually prevents malicious tool usage. The mitigation strategy is to implement the gateway interface first as a mock with validation logic but synthetic tool responses, allowing the rest of the pipeline to integrate against a stable API even if real tool execution is deferred. This stub-first approach ensures that policy decisions can be tested end-to-end without waiting for full tool implementations.

The fifth dependency addresses team coordination and capacity management. The project operates under mixed availability where team members have varying weekly capacity due to other coursework and commitments. Without a regular coordination cadence, blockers remain hidden until they cascade into larger integration problems, ownership of tasks becomes ambiguous, and priorities drift as individuals make local decisions without team alignment. The mitigation is a Monday/Wednesday/Friday coordination cadence with explicit ownership declarations at each checkpoint. This frequent lightweight sync ensures that blockers surface within 48 hours rather than at the end of a sprint, and allows rapid reprioritization when capacity constraints emerge.

The sixth dependency is the baseline run capture process needed for evaluation. The project's value claim is that the security pipeline reduces attack success rate and prevents tool misuse compared to an unprotected baseline. Without baseline measurements, there is no way to demonstrate quantitative improvement or to justify the latency overhead introduced by the security checks. The risk is that the team builds a working pipeline but cannot prove its effectiveness in the evaluation report. The mitigation strategy involves capturing baseline logs and metrics in Week 2 before policy enforcement is fully enabled, ensuring that both configurations baseline and protected are measured with the same scenario set under controlled conditions. This allows direct comparison of attack success rates, false positive rates, and latency overhead.

These six dependencies fall into three risk categories. The high-risk showstoppers, stable development environment, shared interface contracts, and tool gateway stub must be addressed in Week 1 or the project timeline collapses. The medium-risk dependencies test scenario dataset and baseline capture process can begin with minimal implementations and expand incrementally. The process-critical dependency team coordination cadence must be established immediately and maintained consistently throughout the nine-week sprint schedule. If any high-risk dependency fails, the team's priority shifts immediately from feature breadth to integration continuity, focusing on stabilizing the core pipeline before adding advanced capabilities.

4.2 Project Deliverables

The Agentic Security Pipeline project produces five primary deliverables that together constitute a deployable, evaluatable, and extensible security framework for tool-using LLM agents. These deliverables are project-specific technical artifacts rather than coursework assignments, and each addresses a concrete aspect of the threat model described in the project architecture.

The first deliverable is the Agent Security Gateway, a service that mediates both untrusted inputs and tool execution through a unified enforcement boundary. This gateway exposes four core API endpoints: `/ingest` for receiving and normalizing content from untrusted sources, `/score` for risk analysis, `/policy/decide` for deterministic policy evaluation, and `/tools/call` for controlled tool execution. The gateway acts as a chokepoint that prevents any bypass path in the integration, ensuring that all tool calls must flow through policy enforcement. The gateway produces structured audit events for every request, capturing the full decision trace from input normalization through final execution or denial. This deliverable is complete when the API surface is stable, all endpoints return correct response schemas, the gateway correctly blocks unauthorized tool calls, and structured audit logs are written for every operation. The gateway must demonstrate fail-closed behavior, meaning that any validation failure results in denial rather than fallback to unprotected execution.

The second deliverable is the Tool Proxy and Policy Enforcement Layer, which implements the deterministic controls that govern tool usage. This layer enforces several categories of controls: a tool allowlist that blocks any tool not explicitly registered, argument schema validation using Pydantic models to prevent malformed or malicious parameters, privilege level classification (low/medium/high) that determines approval requirements, human approval gates for medium and high-risk operations, and safe fallback behaviors for denied requests. The layer mediates at least three to four distinct tools, demonstrating variation in privilege levels and approval requirements. This deliverable is complete when the system demonstrates correct block and approve behavior for each policy action type, when all tool argument schemas are validated before execution, when unit tests cover policy logic across at least twelve distinct scenarios, and when the approval gate mechanism correctly routes high-risk requests to a human decision interface. The enforcement layer must produce actionable denial messages that explain to the agent why a tool call was blocked and what alternative actions are permitted.

The third deliverable is the Prompt Injection Detection Engine, a hybrid risk scoring system that analyzes untrusted content for four attack categories: instruction override (attempts to replace system instructions), data exfiltration (intent to extract and transmit sensitive information), tool coercion (forced or manipulated tool execution), and obfuscation (encoding tricks or evasion techniques). The engine produces three outputs for every analyzed input: a risk score from 0 to 100 indicating severity, an attack category label identifying the threat type, and a human-readable rationale explaining which signals triggered the score. This deliverable is complete when the normalization layer handles Unicode tricks, zero-width characters, HTML entity decoding, and basic obfuscation

patterns; when the rule baseline achieves low false-positive rates on benign inputs and high true-positive rates on known attack patterns; when documented thresholds map score ranges to policy actions (ALLOW for 0-20, SANITIZE for 21-40, QUARANTINE for 41-70, REQUIRE_APPROVAL for 71-90, BLOCK for 91-100); and when the engine's rationale output provides sufficient detail for debugging false positives and false negatives. The engine must be extensible, allowing new detection rules to be added without requiring changes to the policy or gateway layers.

The fourth deliverable is the Red-Team Harness and Scenario Pack, an automated testing framework that measures the pipeline's effectiveness against both benign and adversarial inputs. The harness implements repeatable scenario execution where the same input set runs against both a baseline configuration (minimal security checks) and a protected configuration (full pipeline enforcement), enabling direct comparison of attack success rate, false positive rate, and latency overhead. The scenario pack initially includes twelve scenarios covering three benign task paths (normal information retrieval, multi-step reasoning, legitimate tool usage), three suspicious paths (ambiguous intent, edge-case phrasing, borderline policy violations), and six adversarial paths (direct prompt injection, indirect injection via retrieved content, tool coercion attempts, data exfiltration through side channels, obfuscated attacks, multi-stage exploitation). This deliverable is complete when the harness can be invoked with a single command (`make eval` or `python -m eval.run`), when all metrics are calculated automatically and written to a structured results table, when both configurations produce comparable audit logs for analysis, and when the evaluation report documents both successful blocks and false positives with specific examples. The harness must support rapid iteration, allowing new scenarios to be added by simply providing input JSON and expected outcome labels.

The fifth deliverable is the Deployable Open-Source Reference Implementation, a complete runnable system with configuration templates, deployment scripts, and comprehensive documentation. This implementation includes a Docker Compose setup that brings up the entire pipeline with a single command, configuration templates for policy thresholds and tool allowlists that can be customized without code changes, reproducible evaluation scripts that run the full scenario pack and generate metrics, and security disclaimers explaining the threat model and known limitations. The reference implementation is complete when a new team member can clone the repository, follow the README instructions, and successfully execute all three demonstration paths (benign allow, suspicious quarantine, malicious block) within thirty minutes without requiring additional guidance. The documentation must clearly explain the sandboxing model, describe how to add custom detection rules, provide a policy threshold tuning guide, and list known limitations such as false positive scenarios and unaddressed attack vectors. This deliverable serves as both the project's primary technical artifact and as a foundation for future extensions, such as integration with real LLM agent frameworks, addition of more sophisticated detection models, or deployment to production environments with real tools and approval workflows.

Together, these five deliverables constitute a complete proof-of-concept that validates the policy-mediated security architecture. The gateway and enforcement layer demonstrate that hard trust boundaries can be implemented without requiring modifications to the agent itself. The detection engine shows that hybrid rule-based scoring can identify threats with acceptable false positive rates. The red-team harness provides quantitative evidence of effectiveness through baseline comparison. The deployable reference implementation ensures that the architecture can be reproduced, extended, and adapted to different agent frameworks. Each deliverable has explicit acceptance criteria that define completion objectively, preventing scope creep and ensuring that the nine-week implementation timeline remains achievable. The project prioritizes working code and reproducible measurements over theoretical completeness, accepting that certain advanced capabilities such as machine learning-based detection, containerized tool sandboxing, and multi-agent coordination will be deferred to future phases.

Chapter 5. Project Architecture

5.1 Architectural Overview

The prototype uses a policy-mediated security pipeline that separates untrusted content processing from privileged tool execution. The architecture is designed so that the model does not make final execution decisions; however, the policy-mediated pipeline contains multiple layers and gateways that allow the final decision to be made in a deterministic fashion.

Core components:

1. Ingest/Normalize
2. Risk Engine
3. Policy Engine
4. Tool Gateway Stub
5. Audit/Telemetry

5.2 Trust Boundaries

- Boundary A (Untrusted Content Boundary)
  - Any externally retrieved content is automatically deemed as untrusted
  - Content must pass through normalization and the risk scoring criteria before it is used within the LLM's context.
- Boundary B (Privileged Execution Boundary)
  - Any action with side effects, such as tool calls, file writes, or API actions, must pass gateway checks.
  - Direct execution is not allowed outside the gateway mediation system.

5.3 High-Level Data Flow

Figure 1.

Input is received and automatically ingested and normalized. It is then passed to the risk engine, which contains multiple layers of defense protocols and strategies. The data is then passed to the policy engine, which contains many rules that allow or disallow certain actions and content. The data is then classified to be either requiring approval or to be quarantined or blocked.

Once it is deemed to require approval, it is then sent to the human approval path, and from there it is sent to the tool gateway stub, which then makes the final decision to deny

or to make the tool action. Subsequently, all of the output of the above gets tracked within the audit telemetry system.

Chapter 6. Evaluation Methodology

[

Describe your evaluation methodology, including testbed-setup, test parameters, datasets used in the evaluation, baseline approaches, and performance measures.

Execute some baseline experiments using baseline implementations you identified in Chapter 3. If code was not available, implement the baseline method and execute experiments with it.

Evaluated: Workbook Assignment 2

]

Chapter 7. System Design/Methodology

Solution Overview

[

Describe your project architecture using diagrams/pseudo-code and a high-level description. If applicable, describe system architecture and details of each component, or include necessary algorithmic details that will affect system performance.

Start implementation of your proposed solution. Document implementation details necessary for others to reproduce your work. Provide tutorial-like documentation for deploying/using your solution.

Evaluated: Workbook Assignment 2

]

Chapter 8. Implementation Plan and Progress

The primary goal is to deliver a working end-to-end prototype from March 4 to May 8, 2026 (nine weeks) that demonstrates the policy-mediated security architecture. The prototype validates that five core pipeline components integrate deterministically and security decisions propagate correctly through audit telemetry.

Five key deliverables define project completion. The first deliverable is an Agent Security Gateway providing a service interface that mediates both untrusted inputs and tool execution, with clear APIs for ingest, scoring, policy decision, and tool calling, functioning as an enforced chokepoint with structured audit events. The second deliverable is a Tool Proxy and Policy Enforcement Layer providing gatekeeper controls including tool allowlists, argument schema validation, privilege levels, human approval gates for medium/high risk, and safe fallbacks (block, sanitize, quarantine) with demonstrable block/approve behavior and unit tests. The third deliverable is a Prompt Injection Detection Engine providing hybrid risk scoring with structured outputs: risk score, attack category label, and human-readable rationale; including normalization, rule baseline, and documented thresholds. The fourth deliverable is a Red-Team Harness and Scenario Pack with automated tests for benign and attack tasks, measuring Attack Success Rate, False Positive Rate, and latency overhead. The fifth deliverable is a Deployable Open-Source Reference Implementation with Docker setup, configuration templates, reproducible evaluation scripts, and security disclaimers.

8.1 Team Structure and Ownership

Four team members drive the implementation with clear ownership and dependencies. Haroon Razzack (Architecture/Integration Lead) is responsible for module interface contracts, integration orchestration, dependency graph, version control of integration harness, and resolution of cross-module issues. Dao Seckar (Detection/Risk) owns risk scoring logic, threat pattern detection, score schema and output contract, rule-based pattern matching, test fixtures for attack patterns, and benchmark setup for risk engine performance. Nkono Andrew Mwase (Policy/Gateway) owns policy action mapping, deterministic threshold logic, tool allowlist and argument validation, approval path logic, demo path validation paths, safe fallback behaviors, and audit logging schema. Kyle Glover (Evaluation/Documentation) owns scenario matrix design, red-team harness implementation, metric computations (ASR, FPR, latency), baseline vs protected comparison runs, evidence collection and documentation, workbook chapter updates, and team coordination notes.

8.2 Core Technology Stack

8.3.1 Runtime and Framework

Python 3.11+ provides modern language features, async/await syntax built-in, and strong security analysis libraries. FastAPI is chosen for automatic request validation, OpenAPI

documentation generation, and efficient async handling. Pydantic provides typed models and automatic JSON schema generation, ensuring all pipeline data conforms to contracts.

8.2.2 Testing and Validation

Pytest is the testing framework with parameterized test cases, parallel execution, and detailed introspection. The test suite minimum is 12 tests (3 ingest, 3 risk, 2 policy, 2 gateway, 2 end-to-end) with a target of 35+ tests covering all critical paths, edge cases, and failure scenarios.

8.2.3 Risk Analysis and Detection

The risk engine uses Python's re module with compiled patterns for attack signatures. Heuristic scoring converts pattern matches into a 0-100 risk score. Four attack categories are detected: instruction override, data exfiltration, tool coercion, obfuscation.

8.2.4 Logging and Audit

Standard Python logging supports development output. Structured NDJSON (newline-delimited JSON) logs are used for audit records, enabling independent parsing and efficient ingestion into log aggregation platforms.

8.3 Acceptance Criteria and Requirements

8.3.1 Functional Acceptance Checklist

The prototype is functionally complete when seven criteria are satisfied. The pipeline accepts at least two source types (DIRECT_PROMPT and RETRIEVED_CONTENT), with source type tracked through the audit trail. Ingest/normalize applies deterministic normalization steps (Unicode canonicalization, whitespace handling, HTML entity decoding, zero-width character removal) with bit-identical output on repeated runs. Risk scoring returns structured output: numeric score (0-100), category labels (INSTRUCTION_OVERRIDE, DATA_EXFILTRATION, TOOL_COERCION, OBFUSCATION, BENIGN), matched signals, and human-readable rationale.

The policy engine deterministically maps risk to a single action from the closed set: ALLOW, SANITIZE, QUARANTINE, REQUIRE_APPROVAL, BLOCK. The mapping is explicit, threshold-based, and reproducible. The tool gateway stub is the only execution path for all target tools; mock tools cannot be directly invoked. The gateway correctly rejects malformed tool calls (missing arguments, wrong types) and unauthorized tool names, returning explicit denial messages and generating audit records. Audit logging records the complete trace per request: request ID, timestamp, source type, normalized content, risk score and signals, policy action and reason, gateway decision and tool status, all in valid JSON format.

8.3.2 End-to-End Demo Acceptance

Three demonstration paths must execute successfully. The benign path (clean low-risk input) results in ALLOW policy action with tool execution and full telemetry. The suspicious path (medium-risk content) results in QUARANTINE or REQUIRE_APPROVAL with denial and rationale logged. The malicious path (high-risk

injection attempt) results in BLOCK with immediate termination, reason logged, and no tool execution attempted.

8.3.3 Testing Acceptance

A minimum of 12 tests must pass: 3 for ingest/normalize, 3 for risk scoring, 2 for policy mapping, 2 for gateway validation, 2 for end-to-end flow. Target of 35+ tests covering all attack categories, edge cases (empty input, very long strings, Unicode variants), error handling, and failure injection. Test run command and output must be documented for reproducibility.

8.3.4 Quality and Handoff Acceptance

Environment setup must be reproducible from documented steps without oral context. Module interfaces must be documented and versioned. All code must have clear docstrings and comments on security-critical sections. Known limitations must be documented with planned mitigations. Test coverage, architecture diagram, threat model, and evaluation results must all be documented in the workbook.

Chapter 9. Project Schedule

Tentative Weekly Sprint Plan: March 2 to May 8, 2026

The implementation follows a nine-week sprint schedule from March 2, 2026 (Week 1) to May 8, 2026 (Week 9). Each sprint is one week, running Monday through Friday, with weekly demos and progress reviews on Fridays.

Week 1: Interfaces and Architecture (March 2–8, 2026)

Sprint 1 focuses on establishing stable architecture contracts and producing an integrated skeleton where all five modules are wired together. Haroon (Architecture/Integration Lead) finalizes interface contracts for all modules, builds an integration harness for the linear flow, and delivers a dependency graph and handoff checklist. Dao (Detection/Risk) defines the risk category taxonomy, implements initial pattern matching rules for the four attack categories, and creates six unit test fixtures (three benign, three malicious). Nkono (Policy/Gateway) implements policy action mapping thresholds and returns gateway validation structures with deny responses and fail-closed behavior. Kyle (Evaluation/Documentation) creates a scenario matrix with benign, suspicious, and malicious paths, establishes a weekly evidence template, and captures baseline outputs.

Gate 1 criteria (Friday): All module contracts are signed off and integration skeleton runs. Failure to meet Gate 1 requires an emergency escalation.

Week 2: Gateway and Policy Implementation (March 9–15, 2026)

Sprint 2 implements the Tool Gateway and Policy Enforcement Layer. Nkono extends gateway validation to include allowlist checking, argument schema validation using Pydantic, approval requirement detection, and deny-path responses with explicit rationale. Nkono also implements safe fallback behaviors when gateway validation fails. Dao refines risk scoring rules based on initial test results and expands the test fixture set. Haroon integrates policy engine outputs with gateway inputs and resolves interface mismatches. Kyle runs the scenario pack through the integrated pipeline and captures evidence screenshots of all three demo paths.

Deliverable update: Tool Proxy and Policy Enforcement Layer takes shape with working allowlist and schema validation.

Week 3: Risk Engine Baseline and Core Module Completion (March 16–22, 2026)

Sprint 3 completes core module implementation. Dao finalizes the risk engine with comprehensive pattern detection across all four attack categories, ensuring low false-positive rates on benign inputs and high true-positive rates on attack inputs. Scoring

logic is tested against 12+ fixture cases covering edge cases (empty input, very long input, encoded attacks, etc.). Haroon locks the integration harness version and documents all module contracts. Nkono implements the audit logger, writing structured NDJSON output with all required fields (request ID, timestamp, scores, actions, decisions). Kyle establishes baseline metrics for latency, memory usage, and audit log volume.

Gate 2 criteria (Friday): Ingest, scoring, policy, gateway, and audit modules are implemented at functional level (not stub-only). All components have unit test coverage.

Week 4: End-to-End Integration and Demo Paths (March 23–29, 2026)

Sprint 4 focuses on end-to-end integration and demonstrating the three required paths. The full pipeline is wired end-to-end, and all components communicate correctly through their defined contracts. Haroon orchestrates integration and resolves any remaining interface or timing issues. Dao and C verify that the three demo paths (benign allow, untrusted quarantine, malicious block) execute correctly with expected outcomes. Kyle runs a comprehensive test suite, captures demo evidence (logs, screenshots, response traces), and documents the run notes with timestamps and inputs.

Gate 3 criteria (Friday): All three demo paths pass: benign input results in ALLOW and tool execution; suspicious input results in QUARANTINE or REQUIRE_APPROVAL with denial; malicious input results in BLOCK with audit recorded. Full end-to-end response trace is captured for each path.

Week 5: Testing, Audit Compliance, and Scenario Expansion (March 30–April 5, 2026)

Sprint 5 ensures comprehensive testing and audit compliance. The Pytest suite is expanded to 35+ test cases with at least 12 required minimum tests (3 ingest, 3 risk, 2 policy, 2 gateway, 2 end-to-end) plus additional edge cases, failure modes, and integration scenarios. Kyle expands the scenario pack from 8 to 12 scenarios covering additional edge cases and adversarial inputs. Audit logging is verified to produce valid JSON and to support end-to-end traceability by request ID. All pipelines are tested with failure injection (e.g., invalid inputs, missing fields, malformed JSON) to verify safe defaults.

Coverage target: 100% of critical code paths tested, with demonstrated pass/fail behavior for all acceptance criteria.

Week 6: Baseline vs Protected Comparison (April 6–12, 2026)

Sprint 6 establishes the evaluation harness and baseline metrics. Kyle prepares two pipeline configurations: one with minimal security checks (baseline) and one with full gateway, risk, and policy enforcement (protected). Run scripts are created to execute the same scenario set against both configurations with results captured in structured tables. Metrics are measured for all 12+ scenarios: Attack Success Rate (ASR) for each attack type, False Positive Rate (FPR) on benign inputs, approval burden (number of approvals per benign task), and latency overhead. A detailed results table is produced showing baseline vs protected performance.

Week 7: Evaluation Report and Metrics Analysis (April 13–19, 2026)

Sprint 7 completes the evaluation and produces a comprehensive report. Kyle analyzes all collected metrics and produces a results table (scenarios, outcomes, metrics, rationales). Both successful blocks and false positives are documented with examples. The report includes narrative analysis of policy threshold tuning opportunities based on the results. Known limitations are documented (e.g., false positives in certain scenario classes), and recommendations for future detection improvements are provided.

Gate 4 criteria (Friday): Baseline vs protected comparison runs reproducibly. Evaluation report with ASR, FPR, and latency metrics is complete and peer-reviewed.

Week 8: Documentation, Deployment, and Reproducibility (April 20–26, 2026)

Sprint 8 ensures the prototype is deployment-ready and fully documented. Local setup instructions (venv + pip) are verified on a clean machine. Docker setup (docker-compose up --build) is tested and verified. Environment variables and configuration are documented. The README is updated with all required sections: what this is, prerequisites, setup instructions for both local and Docker, test running, demo path execution, audit log reading instructions, project structure overview, how the pipeline works, policy threshold table, available tools, adding custom rules, and sandboxing model. Known limitations are clearly listed with next steps. All team handoff materials are finalized.

Quality criteria: A team member unfamiliar with the project can reproduce the entire setup and run the demo paths using only the README.

Week 9: Final Integration, Regression Testing, and Workbook Handoff (April 27–May 8, 2026)

Sprint 9 is the final integration and delivery sprint. All components are regression tested to ensure no regressions from previous sprints. The complete test suite (35+ tests) is run to completion with all tests passing. Demo paths are verified one final time. The workbook chapters corresponding to implementation (chapters 4, 5, 8, 9) are updated with actual progress data, schedule variance notes, team contributions, and lessons learned. Final evidence collection includes: test run outputs, demo screenshots, audit log examples, metrics tables, and deviation notes from original plan. Code is prepared for handoff with clear documentation, docstrings, and inline comments explaining security-critical sections.

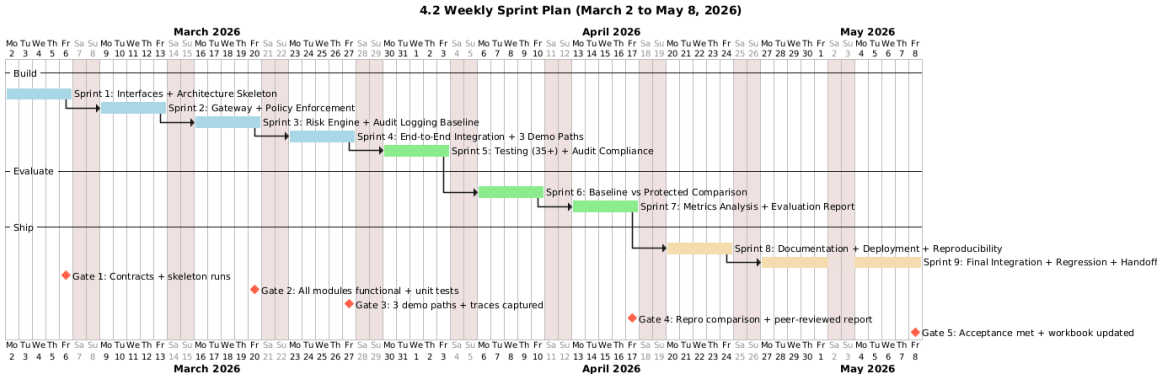
Gate 5 criteria (Friday May 8): All acceptance criteria are met. Chapters 4, 5, 8, and 9 are updated with implementation evidence. Prototype is ready for hand-off or for pivot to integration with real LLM agents.

The tool gateway stub is the only path through which the prototype can execute side-effecting operations. A small set of mock tools (summarize, fetch_url, write_note, search_notes) are stubbed to return synthetic data without making real network calls or file writes. The gateway validates tool name against an allowlist and arguments against a Pydantic schema, enforces the policy decision, and returns execution or denial with reason. This stub approach allows validation of the gateway mechanism without requiring real tool implementations or network access.

The audit logging component writes structured NDJSON output to audit_logs/audit.ndjson, with one JSON object per request. Each log entry includes request ID, timestamp, source type, normalized content hash, risk score and categories, policy action and reason, gateway decision, and tool output status. This audit trail is designed to support incident reconstruction, false-positive analysis, and evaluation metrics across red-team testing campaigns.

Sprint Timeline Visualization

The following Gantt-style visualization shows the dependency structure and critical path:



References

[1] OWASP Foundation, "OWASP Top 10 for LLM Applications 2025," ver. 2025 (v2.0), Nov. 2024. [Online]. Available: [https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf] (https://owasp.org/www-project-top-10-for-large-language-model-applications/assets/PDF/OWASP-Top-10-for-LLMs-v2025.pdf). Accessed: Mar. 4, 2026. (OWASP Foundation[1])

[2] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), Copenhagen, Denmark, Nov. 2023, pp. 79–90, doi: 10.1145/3605764.3623985. (ACM Digital Library[2])

[3] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," in Proc. 33rd USENIX Security Symp. (USENIX Security '24), 2024. [Online]. Available: https://www.usenix.org/system/files/usenixsecurity24-liu-yupei.pdf. Accessed: Mar. 4, 2026. (USENIX[3])

[4] S. Schulhoff et al., "Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs through a Global Scale Prompt Hacking Competition," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP 2023), Singapore, Dec. 2023, pp. 4945–4977, doi: 10.18653/v1/2023.emnlp-main.302. (ACL Anthology[4])

[5] P. Wang et al., "The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis," *arXiv* preprint arXiv:2602.10453, 2026. [Online]. Available: https://arxiv.org/abs/2602.10453. Accessed: Mar. 4, 2026. (arXiv[5])

[6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of the Assoc. for Computational Linguistics: ACL 2024*, Bangkok, Thailand, Aug. 2024. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/. Accessed: Mar. 4, 2026. (ACL Anthology[6])

[7] H. Zhang *et al*., "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," in *Proc. Int. Conf. on Learning Representations*

(ICLR 2025), 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5750f91d8fb9d5c02bd8ad2c3b44456b-Abstract-Conference.html. Accessed: Mar. 4, 2026. (ICLR Proceedings[7])

[8] Y. Kaya, A. Landerer, S. Pletinckx, M. Zimmermann, C. Kruegel, and G. Vigna, "When AI Meets the Web: Prompt Injection Risks in Third-Party AI Chatbot Plugins," *arXiv* preprint arXiv:2511.05797, 2025 (accepted to IEEE S&P 2026 per authors' version). [Online]. Available: https://arxiv.org/abs/2511.05797. Accessed: Mar. 4, 2026. (arXiv[8])

[9] Y. Ruan *et al*., "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in *Proc. Int. Conf. on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/7274ed909a312d4d869cc328ad1c5f04-Abstract-Conference.html. Accessed: Mar. 4, 2026. (ICLR Proceedings[9])

[10] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and Benchmarking Prompt Injection Attacks and Defenses," *arXiv* preprint arXiv:2310.12815, 2023. [Online]. Available: https://arxiv.org/abs/2310.12815. Accessed: Mar. 4, 2026. (arXiv[10])

[11]. J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models," *arXiv* preprint arXiv:2312.14197, 2023. [Online]. Available: https://arxiv.org/abs/2312.14197. Accessed: Mar. 4, 2026. (arXiv[11])

[12] Y. Chen, H. Li, Y. Sui, Y. He, Y. Liu, Y. Song, and B. Hooi, "Can Indirect Prompt Injection Attacks Be Detected and Removed?" in *Proc. 63rd Annu. Meeting Assoc. for Computational Linguistics (ACL 2025)*, Vienna, Austria, Jul. 2025, pp. 18189–18206, doi: 10.18653/v1/2025.acl-long.890. (ACL Anthology[12])

[13] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions," *arXiv* preprint arXiv:2404.13208, 2024. [Online]. Available: https://arxiv.org/abs/2404.13208. Accessed: Mar. 4, 2026. (arXiv[13])

[14] S. Chen, Y. Wang, N. Carlini, C. Sitawarin, and D. Wagner, "Defending Against Prompt Injection With a Few DefensiveTokens," in *Proc. Workshop on Artificial Intelligence and Security (AISec '25)*, Taipei, Taiwan, Oct. 2025, pp. 242–252, doi: 10.1145/3733799.3762982. (arXiv[14])

[15] S. M. A. Hossain *et al*., "A Multi-Agent LLM Defense Pipeline Against Prompt Injection Attacks," *arXiv* preprint arXiv:2509.14285, 2025. [Online]. Available: https://arxiv.org/abs/2509.14285. Accessed: Mar. 4, 2026. (arXiv[15])

[16] C. Anil *et al*., "Many-shot Jailbreaking," in *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/ea456e232efb72d261715e33ce25f208-Paper-Conference.pdf. Accessed: Mar. 4, 2026. (NeurIPS Proceedings[16])

[17] T. Yuan *et al*., "R-Judge: Benchmarking Safety Risk Awareness for LLM Agents," in *Findings of the Assoc. for Computational Linguistics: EMNLP 2024*, Miami, FL, USA, Nov. 2024, pp. 1467–1490, doi: 10.18653/v1/2024.findings-emnlp.79. (ACL Anthology[17])

[18] S. Datta, S. K. Nahin, A. Chhabra, and P. Mohapatra, "Agentic AI Security: Threats, Defenses, Evaluation, and Open Challenges," *arXiv* preprint arXiv:2510.23883, 2025. [Online]. Available: https://arxiv.org/abs/2510.23883. Accessed: Mar. 4, 2026. (arXiv[18])

[19] M. Bhatt *et al*., "Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models," *arXiv* preprint arXiv:2312.04724, 2023. [Online]. Available: https://arxiv.org/abs/2312.04724. Accessed: Mar. 4, 2026. (arXiv[19])

[20] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html. Accessed: Mar. 4, 2026. (NeurIPS Proceedings[20])

[21] Protect AI, "LLM Guard," 2024. [Online]. Available: https://protectai.com/llm-guard. Accessed: Mar. 4, 2026. (Protect AI[21])

[22] Protect AI, "Rebuff (LLM Prompt Injection Detector)," 2023. [Online]. Available: https://github.com/protectai/rebuff. Accessed: Mar. 4, 2026. (GitHub[22])

[23] Lakera, "Lakera Guard," 2024. [Online]. Available: https://docs.lakera.ai/guard. Accessed: Mar. 4, 2026. (Lakera API Documentation[23])

[24] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, “NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails,” in *Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP 2023 Demo)*, Singapore, Dec. 2023, pp. 431–445. [Online]. Available: https://aclanthology.org/2023.emnlp-demo.40/. Accessed: Mar. 4, 2026. (ACL Anthology[24])

[25] Anthropic, "Security - Claude Code Docs," 2026. [Online]. Available: https://code.claude.com/docs/en/security (Claude[1])

[26] Cursor, "Implementing a secure sandbox for local agents," 2026. [Online]. Available: https://cursor.com/blog/agent-sandboxing; Cursor, "Hooks for security and platform teams," 2026. [Online]. Available: https://cursor.com/blog/hooks-partners (Cursor[2])

[27] I. Habler, E. Huang, and O. Antebi, "Agentic IDEs under fire: Dissecting the real CVEs that exposed Cursor, Windsurf, and Void," *Medium*, 2025. [Online]. Available: https://idanhabler.medium.com/agentic-ides-under-fire-dissecting-the-real-cves-that-exposed-cursor-windsurf-and-void-bd4fae316777 (Medium[3])

[28] AIM Security, "CurXecute: RCE in Cursor via MCP auto-start," 2025. [Online]. Available: https://www.aim.security/post/when-public-prompts-turn-into-local-shells-rce-in-cursor-via-mcp-auto-start (Tenable®[4])

[29] K. Aizen, "AI coding agent attack surface: A full taxonomy," *SnailSploit*, 2026. [Online]. Available: https://snailsploit.com/ai-security/ai-coding-agent-attack-surface/; TheHackerWire, "OpenCode Agent RCE via unauthenticated local server (CVE-2026-22812)," 2026. [Online]. Available: https://www.thehackerwire.com/opencode-agent-rce-via-unauthenticated-local-server/ (SnailSploit[5])

[30] OpenHands, "Mitigating prompt injection attacks in software agents," 2025. [Online]. Available: https://openhands.dev/blog/mitigating-prompt-injection-attacks-in-software-agents (openhands.dev[6])

[31] NVIDIA AI Red Team, "Practical security guidance for sandboxing agentic workflows and managing execution risk," 2026. [Online]. Available: https://developer.nvidia.com/blog/practical-security-guidance-for-sandboxing-agentic-workflows-and-managing-execution-risk/ (NVIDIA Developer[7])

[32] Vercel, "Security boundaries in agentic architectures," 2026. [Online]. Available: https://vercel.com/blog/security-boundaries-in-agentic-architectures (Vercel[8])

[33] S. Willison, "The lethal trifecta," 2025. [Online]. Available: https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/ (simonwillison.net[9])

[34] OWASP Foundation, "OWASP Top 10 for LLM Applications," Version 2.0, 2025. [Online]. Available: https://genai.owasp.org/llm-top-10/

[35] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90.

[36] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. USENIX Security Symposium, 2024.

[37] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si, S. Bharadwaj, B. Dolan-Gavitt, H. Kiyani, and J. Boyd-Graber, "Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global scale prompt hacking competition," in Proc. 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[38] OWASP Foundation, "LLM06: Excessive Agency," in OWASP Top 10 for LLM Applications, Version 2.0, 2025. [Online]. Available: https://genai.owasp.org/llm-top-10/